Seone Frontend Documentation

> **Status:** Complete | **Last Updated:** 2026-01-19
> **Audience:** Developers new to the codebase, from freshers to senior engineers

---

What is Seone?

Seone is an **AI-powered video processing platform** that transforms YouTube videos into short-form content clips. The frontend is the user-facing application that allows users to:

1. **Submit YouTube URLs** for processing
2. **Configure clip parameters** (duration, count, templates, language)
3. **Monitor job progress** in real-time via WebSocket
4. **Download generated clips** when processing completes

Simple Explanation

Think of Seone as a "video factory control panel." You give it a YouTube link, tell it how many clips you want and how long they should be, pick a template style, and hit submit. The backend does the heavy lifting (downloading, transcribing, analyzing, rendering), while the frontend shows you what's happening and displays your clips when they're ready.

Technical Explanation

The frontend is a **Next.js 16 App Router application** using: - **React 19** for UI - **Zustand** for state management - **Framer Motion** for animations - **CSS Modules** for styling - **WebSocket + REST polling** for real-time updates

Why This Matters in Production

The frontend is the **only part users see**. If it fails to show job progress, users think the system is broken. If it doesn't handle auth properly, they can't access their jobs. If loading states are missing, the UI feels janky. This documentation ensures engineers understand exactly how every piece works.

Documentation Structure

This documentation follows a **production-grade hierarchy**:

Folder	Purpose
`00_overview/`	This file — high-level introduction
`01_frontend_architecture/`	Technology stack, patterns, design decisions
`02_repository_map/`	Complete file-by-file inventory
`03_routing_and_navigation/`	Next.js App Router structure, route groups
`04_state_management/`	Zustand stores, data flow, persistence
`05_api_and_backend_integration/`	REST endpoints, request/response contracts
`06_real_time_updates/`	WebSocket implementation, reconnection, polling
`07_authentication_and_sessions/`	Google OAuth, JWT handling, session management
`08_ui_components_and_design_system/`	Component library, CSS variables, animations
`09_error_handling_and_edge_cases/`	Error states, recovery, user feedback
`10_performance_and_loading_states/`	Loading indicators, skeleton screens, optimizations
`11_environment_and_configuration/`	Environment variables, config management
`12_accessibility_and_ux_decisions/`	A11y considerations, UX patterns
`13_known_limitations_and_risks/`	Current gaps, known issues, future improvements

Quick Start for New Developers

1. Installation

```
cd Seone-Frontend
npm install
```

2. Environment Setup

Create `.env.local` with:

```
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_WS_URL=ws://localhost:8000
NEXT_PUBLIC_GOOGLE_CLIENT_ID=your-google-client-id
NEXT_PUBLIC_DATA_URL=http://localhost:8000/data
```

3. Run Development Server

```
npm run dev
```

The app will be available at `http://localhost:3000`.

Key User Flows

1. Login Flow

```
Landing Page → Login Page → Google OAuth → Token Exchange → Dashboard
```

- Only `@creativefuel.io` domain is allowed
- JWT stored in secure cookie
- User info persisted in sessionStorage via Zustand

2. Job Creation Flow

```
Dashboard → New Job Page → Fill Form → Select Templates → Submit → Job Detail Page
```

- YouTube URL validation happens client-side
- Form data validated before submission
- On success, redirected to job detail page with WebSocket connection

3. Job Monitoring Flow

```
Job Detail Page → WebSocket Connect → Receive Events → Update UI → REST Reconciliation
```

- WebSocket provides real-time step/clip updates
- REST polling every 3 seconds ensures state convergence
- On job completion, final REST fetch reconciles all data

Frontend-Backend Contract Summary

Frontend Action	Backend Endpoint	Key Contract
Login	`POST /api/v1/auth/google?token=`	Returns `access_token`
Get User	`GET /api/v1/auth/me`	Returns `User` object
List Jobs	`GET /api/v1/jobs?page=&per_page=`	Returns paginated job list
Create Job	`POST /api/v1/jobs`	Returns `{ id, ws_url }`
Get Job	`GET /api/v1/jobs/{id}`	Returns full job details
WebSocket	`WS /ws/jobs/{id}?token=`	Real-time events

See `05_api_and_backend_integration/` for complete contracts.

State Contracts

The frontend has documented invariants in `CONTRACTS.md`:

Invariant	Guarantee
`items: Job[]`	Always an array, never undefined
`job: Job | null`	Null is valid transitional state; components must handle
`liveClips: Clip[]`	Always an array
Store reset	Synchronous; components will render during reset→fetch gap

Violations of these contracts are bugs, not expected behavior.

Next Steps

1. Start with `01_frontend_architecture/` to understand the overall structure
2. Read `03_routing_and_navigation/` to understand URL patterns
3. Study `04_state_management/` before touching any stores
4. Review `06_real_time_updates/` before modifying WebSocket logic

Frontend Architecture

> **Confidence:** CONFIRMED | **Source:** Direct code analysis

---

Technology Stack

Core Framework

Technology	Version	Purpose
Next.js	16.1.1	App Router, SSR, routing
React	19.2.3	UI components, hooks
TypeScript	^5	Type safety

State Management

Technology	Version	Purpose
Zustand	^5.0.10	Global state management

Styling & Animation

Technology	Version	Purpose
CSS Modules	(built-in)	Scoped component styles
Framer Motion	^12.26.1	Animations and transitions
clsx	^2.1.1	Conditional class names

Authentication

Technology	Version	Purpose
@react-oauth/google	^0.13.4	Google OAuth integration
js-cookie	^3.0.5	Cookie management for JWT

Architecture Diagram

```
┌──────────────────────────────────────────────────────────────┐
│                        NEXT.JS APP ROUTER                    │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  ┌──────────┐    ┌──────────────┐    ┌──────────────────┐   │
│  │  ROUTES  │    │   LAYOUTS    │    │      PAGES       │   │
│  │ (auth)   │──→│ GoogleOAuth  │──→│ login/page.tsx   │   │
│  │ (dashboard)│  │ AuthGuard    │    │ dashboard/page.tsx │  │
│  │          │    │ AppShell     │    │ jobs/page.tsx    │   │
│  │          │    │              │    │ jobs/[id]/page.tsx │  │
│  │          │    │              │    │ new/page.tsx     │   │
│  └──────────┘    └──────────────┘    └──────────────────┘   │
│                                                              │
├──────────────────────────────────────────────────────────────┤
│                        COMPONENT LAYER                       │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐       │
│  │    LAYOUT    │  │     JOB      │  │     UI       │       │
│  ├──────────────┤  ├──────────────┤  ├──────────────┤       │
│  │ AppShell     │  │ ClipGallery  │  │ Button       │       │
│  │ AuthGuard    │  │ JobsList     │  │ Input        │       │
│  │ Sidebar      │  │ PipelineTime.│  │ DualRangeSlid│       │
│  │ TopBar       │  │ SubmitPanel  │  │              │       │
│  │ Inspector    │  │ TemplateSelec│  │              │       │
│  │ PageTransition│ │              │  │              │       │
│  └──────────────┘  └──────────────┘  └──────────────┘       │
│                                                              │
├──────────────────────────────────────────────────────────────┤
│                          DATA LAYER                          │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐       │
│  │    HOOKS     │  │    STORES    │  │   SERVICES   │       │
│  ├──────────────┤  ├──────────────┤  ├──────────────┤       │
│  │ useJobs      │  │ useAuthStore │  │ auth.ts      │       │
│  │ useJobSubmit │  │ useJobStore  │  │ (authFetch)  │       │
│  │ useJobWebSock│  │ useAppStore  │  │              │       │
│  │ usePages     │  │              │  │              │       │
│  └──────────────┘  └──────────────┘  └──────────────┘       │
│                                                              │
├──────────────────────────────────────────────────────────────┤
│                        EXTERNAL SERVICES                     │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  ┌──────────────────────────┐    ┌────────────────────────┐  │
│  │      REST API            │    │       WEBSOCKET        │  │
│  │  /api/v1/auth/*          │    │  /ws/jobs/{id}?token=  │  │
│  │  /api/v1/jobs/*          │    │                        │  │
│  │  /api/v1/pages/*         │    │                        │  │
│  └──────────────────────────┘    └────────────────────────┘  │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Design Patterns

1. Container/Presenter Pattern (Implicit)

Pattern: Pages act as containers that fetch data and pass it to presentational components.

Example:

```
// Page (Container) - src/app/(dashboard)/dashboard/new/page.tsx
export default function NewJobPage() {
    const { formData, errors, state, updateField, togglePage, submit } = useJobSubmit();

    return (
        <SubmitPanel formData={formData} onUpdateField={updateField} />
        <TemplateSelector selectedPages={formData.selectedPages} onToggle={togglePage} />
    );
}

// Component (Presenter) - receives data via props, renders UI
export function SubmitPanel({ formData, onUpdateField }: Props) { ... }
```

2. Custom Hooks for Data Fetching

Pattern: All data fetching is encapsulated in custom hooks.

Hook	Responsibility
`useJobs()`	Fetch paginated job list, handle pagination
`useJobSubmit()`	Form state, validation, job creation
`useJobWebSocket()`	WebSocket connection, event handling, polling
`usePages()`	Fetch available templates with caching

Why: Separates data logic from UI logic. Components stay focused on rendering.

3. Zustand Store Pattern

Pattern: Minimal, focused stores with clear contracts.

```
// src/stores/job.ts - Single responsibility: job detail state
interface JobState {
    job: Job | null;        // Current job being viewed
    liveClips: Clip[];      // Clips received via WebSocket
    wsConnected: boolean;   // WebSocket connection status
    // ... actions
}
```

Key Principle: Stores provide type-accurate defaults: - Objects → `null` - Arrays → `[]` - Booleans → explicit `true/false`

Never: Use mock objects as defaults (masks bugs).

4. REST + WebSocket Hybrid

Pattern: REST for initial load and reconciliation; WebSocket for real-time updates.

```
1. Page Mount      → REST fetch (authoritative snapshot)
2. WebSocket Open  → Subscribe to events
3. Events Arrive   → Update local state incrementally
4. Job Completed   → REST reconciliation (final authority)
5. Polling         → Every 3s while job is active (redundancy)
```

Why: WebSocket can miss events (late connect, disconnect). REST ensures eventual consistency.

Directory Structure

```
src/
├── app/                    # Next.js App Router pages
│   ├── (auth)/             # Route group: public auth pages
│   │   ├── layout.tsx      # Wraps children in GoogleOAuthProvider
│   │   └── login/
│   ├── (dashboard)/        # Route group: protected dashboard pages
│   │   ├── layout.tsx      # Wraps children in AuthGuard + AppShell
│   │   └── dashboard/
│   ├── globals.css         # Design system, CSS variables
│   ├── layout.tsx          # Root layout (fonts, PageTransition)
│   └── page.tsx            # Landing page
│
├── components/
│   ├── job/                # Job-related components
│   ├── layout/             # Layout components (shell, sidebar, etc.)
│   └── ui/                 # Reusable UI primitives
│
├── hooks/                  # Custom React hooks
│   ├── useJobs.ts          # Job list fetching
│   ├── useJobSubmit.ts     # Job creation form
│   ├── useJobWebSocket.ts  # WebSocket + polling
│   └── usePages.ts         # Template fetching
│
├── lib/                    # Utilities and config
│   ├── animations.ts       # Framer Motion presets
│   ├── config.ts           # Environment config, endpoints
│   └── utils.ts            # Helper functions
│
├── services/               # API service layer
│   └── auth.ts             # Auth functions, authFetch wrapper
│
├── stores/                 # Zustand stores
│   ├── app.ts              # UI state (sidebar, inspector)
│   ├── auth.ts             # Auth state (user, tokens)
│   └── job.ts              # Job detail state
│
└── types/                  # TypeScript type definitions
    ├── index.ts            # Core types (User, Job, Clip, etc.)
    └── job.ts              # Form-specific types
```

Key Architectural Decisions

1. App Router Route Groups

Decision: Use `(auth)` and `(dashboard)` route groups to apply different layouts.

Rationale: - `(auth)` routes need GoogleOAuthProvider but no auth guard - `(dashboard)` routes need AuthGuard and AppShell - Route groups don't affect URL structure

2. sessionStorage for Auth Persistence

Decision: Auth state persisted to sessionStorage, not localStorage.

Rationale: - Session ends when browser closes (security) - JWT in cookie for API requests (automatic) - User info in sessionStorage for display only

3. CSS Modules over Tailwind

Decision: CSS Modules with a custom design system.

Rationale: - Full control over design tokens - No runtime CSS-in-JS overhead - Better for premium, branded UI

4. Clip Merging Logic

Decision: Complex clip de-duplication in `useJobStore`.

Rationale: - WebSocket sends clips incrementally - REST returns full clip array - Must merge without duplicates - Must maintain sort order by index

```
// src/stores/job.ts
const mergeClips = (base: Clip[], incoming: Clip[]): Clip[] => {
    const byKey = new Map<string, Clip>();
    for (const clip of base) byKey.set(getClipKey(clip), clip);
    for (const clip of incoming) byKey.set(getClipKey(clip), clip);
    return sortClips(Array.from(byKey.values()));
};
```

Performance Considerations

1. **Selective Zustand Subscriptions:** Components select only needed state slices
2. **Staggered Animations:** Lists use stagger containers to prevent frame drops
3. **Pages Cache:** Template list cached in-memory to avoid refetching
4. **Conditional WebSocket:** Only connects for active (non-terminal) jobs
5. **Polling Stops:** When job reaches terminal state (completed/failed)

Repository Map

> **Confidence:** CONFIRMED | **Source:** Direct file system scan

This document provides a **complete inventory** of every file in the Seone Frontend repository.

Root Directory

File	Purpose	Key Contents
`package.json`	Dependencies, scripts	Next.js 16, React 19, Zustand, Framer Motion
`tsconfig.json`	TypeScript config	Strict mode, path alias `@/*` → `./src/*`
`next.config.ts`	Next.js config	Currently empty (defaults)
`eslint.config.mjs`	ESLint config	Next.js ESLint preset
`.prettierrc`	Prettier config	Code formatting rules
`.gitignore`	Git ignore rules	node_modules, .next, .env.local
`CONTRACTS.md`	State contracts	Frontend invariants (normative document)
`README.md`	Project readme	Basic setup instructions

`/src/app` — Pages and Layouts

Root Level

File	Type	Purpose
`layout.tsx`	Server Component	Root layout with Inter/JetBrains fonts, PageTransition wrapper
`page.tsx`	Client Component	Landing page with hero section, feature cards, CTAs
`globals.css`	Stylesheet	Design system: CSS variables, utility classes, animations
`page.module.css`	CSS Module	Landing page styles
`favicon.ico`	Asset	Browser tab icon

`/src/app/(auth)` — Authentication Route Group

File	Type	Purpose
`layout.tsx`	Server Component	Wraps children in `GoogleOAuthProvider`

`/src/app/(auth)/login`

File	Type	Purpose
`page.tsx`	Client Component	Login page with Google OAuth button, domain validation
`page.module.css`	CSS Module	Login page styles

`/src/app/(dashboard)` — Dashboard Route Group

File	Type	Purpose
`layout.tsx`	Server Component	Wraps children in `AuthGuard` + `AppShell`

`/src/app/(dashboard)/dashboard`

File	Type	Purpose
`page.tsx`	Client Component	Dashboard home: welcome message, stats cards, quick actions
`page.module.css`	CSS Module	Dashboard page styles

`/src/app/(dashboard)/dashboard/new`

File	Type	Purpose
`page.tsx`	Client Component	New job page: SubmitPanel + TemplateSelector
`page.module.css`	CSS Module	New job page styles

`/src/app/(dashboard)/dashboard/jobs`

File	Type	Purpose
`page.tsx`	Client Component	Jobs list page: renders `JobsList` component
`page.module.css`	CSS Module	Jobs list page styles

`/src/app/(dashboard)/dashboard/jobs/[id]`

File	Type	Purpose
`page.tsx`	Client Component	Job detail page: REST fetch → WebSocket → PipelineTimeline + ClipGallery
`page.module.css`	CSS Module	Job detail page styles

`/src/components` — Reusable Components

`/src/components/layout` — Layout Components

File	Purpose	Dependencies
`AppShell.tsx`	Main app layout with sidebar, topbar, inspector	TopBar, Sidebar, Inspector, useAppStore
`AppShell.module.css`	AppShell styles	—
`AuthGuard.tsx`	Route protection, redirects based on auth state	useAuthStore, useRouter
`GoogleOAuthProvider.tsx`	Wraps @react-oauth/google Provider	config.auth.googleClientId
`Inspector.tsx`	Right panel showing active job details	useAppStore
`Inspector.module.css`	Inspector styles	—
`PageTransition.tsx`	Framer Motion page transition wrapper	framer-motion, usePathname
`Sidebar.tsx`	Left navigation with nav items, user info	useAuthStore, useAppStore, usePathname
`Sidebar.module.css`	Sidebar styles	—
`TopBar.tsx`	Header bar with logo, user menu, logout	useAuthStore, useAppStore, useRouter
`TopBar.module.css`	TopBar styles	—
`index.ts`	Barrel export	All layout components

`/src/components/job` — Job-Related Components

File	Purpose	Dependencies
`ClipGallery.tsx`	Grid display of generated video clips	useJobStore, getMediaUrl, Button
`ClipGallery.module.css`	ClipGallery styles	—
`JobsList.tsx`	Paginated grid of job cards	useJobs, useRouter, Button
`JobsList.module.css`	JobsList styles	—
`PipelineTimeline.tsx`	Visual step progress indicator	useJobStore
`PipelineTimeline.module.css`	PipelineTimeline styles	—
`SubmitPanel.tsx`	Job submission form (URL, duration, count, language, mode)	Input, DualRangeSlider, Button
`SubmitPanel.module.css`	SubmitPanel styles	—
`TemplateSelector.tsx`	Template/page selection grid grouped by category	usePages
`TemplateSelector.module.css`	TemplateSelector styles	—
`index.ts`	Barrel export	All job components

`/src/components/ui` — UI Primitives

File	Purpose	Dependencies
`Button.tsx`	Button component with variants (primary, secondary, ghost, danger)	framer-motion, animations
`Button.module.css`	Button styles	—
`Input.tsx`	Text input with label, error, icons	—
`Input.module.css`	Input styles	—
`DualRangeSlider.tsx`	Min/max range slider for duration	—
`DualRangeSlider.module.css`	DualRangeSlider styles	—
`index.ts`	Barrel export	Button, Input, DualRangeSlider

`/src/hooks` — Custom React Hooks

File	Purpose	Key Functions
`useJobs.ts`	Fetch paginated job list	`fetchJobs()`, `loadMore()`, `refresh()`
`useJobSubmit.ts`	Job creation form management	`updateField()`, `togglePage()`, `validate()`, `submit()`
`useJobWebSocket.ts`	WebSocket connection + polling + event handling	`connect()`, `fetchJob()`, event handlers
`usePages.ts`	Fetch available templates with in-memory caching	Pages cached globally
`index.ts`	Barrel export	All hooks

`/src/lib` — Utilities and Configuration

File	Purpose	Key Exports
`config.ts`	Environment configuration	`config`, `endpoints`, `getApiUrl()`, `getWsUrl()`, `getMediaUrl()`
`utils.ts`	Helper functions	`cn()`, `formatDuration()`, `formatRelativeTime()`, `isValidYouTubeUrl()`, `debounce()`
`animations.ts`	Framer Motion presets	Variants for pages, modals, lists, buttons, etc.

`/src/services` — API Service Layer

File	Purpose	Key Functions
`auth.ts`	Authentication and token management	`exchangeGoogleToken()`, `getCurrentUser()`, `logout()`, `getAuthToken()`, `setAuthToken()`, `clearAuthToken()`, `authFetch()`, `isAllowedDomain()`, `isTokenExpired()`, `getValidAuthToken()`

`/src/stores` — Zustand Stores

File	Purpose	Key State
`auth.ts`	Authentication state	`user`, `isAuthenticated`, `isLoading` + actions
`job.ts`	Active job state	`job`, `liveClips`, `wsConnected`, `lastEventAt` + actions
`app.ts`	UI state	`isInspectorOpen`, `isSidebarOpen`, `activeJobId` + actions
`index.ts`	Barrel export	All stores

`/src/types` — TypeScript Definitions

File	Purpose	Key Types
`index.ts`	Core API types	`User`, `AuthState`, `AuthResponse`, `Job`, `JobStatus`, `JobPhase`, `Clip`, `Page`, `WebSocketEvent`, `ApiResponse`, `ApiError`
`job.ts`	Form types	`SubmissionFormData`, `FormErrors`, `SubmissionState`, constants (`DURATION_MIN`, `DURATION_MAX`, etc.)

`/public` — Static Assets

File	Purpose
`file.svg`	Generic file icon
`globe.svg`	Globe icon
`next.svg`	Next.js logo
`vercel.svg`	Vercel logo
`window.svg`	Window icon

File Count Summary

Category	Count
Pages/Layouts	12
Components	18
CSS Modules	15
Hooks	5
Stores	4
Services	1
Lib Utilities	3
Types	2
Config Files	6
Total Source Files	**~66**

Routing and Navigation

> **Confidence:** CONFIRMED | **Source:** Direct code analysis of app/ directory

---

Next.js App Router Overview

Seone uses **Next.js 16 App Router** with the following key features:

- **Route Groups:** `(auth)` and `(dashboard)` to apply different layouts without affecting URLs
- **Dynamic Routes:** `[id]` for job detail pages
- **Nested Layouts:** Each route group has its own layout wrapping child routes
- **Client Components:** All pages are client-side rendered (marked with `'use client'`)

URL Structure

URL	Page	Layout Chain	Auth Required
`/`	Landing page	RootLayout → PageTransition	❌ No
`/login`	Login page	RootLayout → AuthLayout (GoogleOAuth)	❌ No
`/dashboard`	Dashboard home	RootLayout → DashboardLayout (AuthGuard + AppShell)	✅ Yes
`/dashboard/new`	Create new job	RootLayout → DashboardLayout	✅ Yes
`/dashboard/jobs`	Jobs list	RootLayout → DashboardLayout	✅ Yes
`/dashboard/jobs/{id}`	Job detail	RootLayout → DashboardLayout	✅ Yes

Route Group: `(auth)`

Purpose: Public authentication pages that need GoogleOAuthProvider but NOT auth guard.

File: `src/app/(auth)/layout.tsx`

```
import { GoogleOAuthProvider } from '@/components/layout/GoogleOAuthProvider';

export default function AuthLayout({ children }) {
    return (
        <GoogleOAuthProvider>
            {children}
        </GoogleOAuthProvider>
    );
}
```

Simple Explanation

This layout wraps the login page so that Google's OAuth scripts are available. Users can log in without being authenticated (obviously — that's the point of a login page).

Technical Explanation

- `GoogleOAuthProvider` initializes the `@react-oauth/google` library
- Passes the Google Client ID from environment config
- All child routes automatically have access to `GoogleLogin` component

Child Routes

Route	File	Purpose
`/login`	`(auth)/login/page.tsx`	Google OAuth login

Route Group: `(dashboard)`

Purpose: Protected dashboard pages that require authentication and use the main app layout.

File: `src/app/(dashboard)/layout.tsx`

```
import { AuthGuard, AppShell } from '@/components/layout';

export default function DashboardLayout({ children }) {
    return (
        <AuthGuard requireAuth={true} redirectTo="/login">
            <AppShell>{children}</AppShell>
        </AuthGuard>
    );
}
```

Simple Explanation

This layout adds a security check: if you're not logged in, you get sent to the login page. If you are logged in, you see the main app with a sidebar and top bar.

Technical Explanation

- `AuthGuard` checks `useAuthStore` for authentication state
- If not authenticated, redirects to `/login`
- If authenticated, renders `AppShell` which includes Sidebar, TopBar, and Inspector
- Children render in the main content area

Child Routes

Route	File	Purpose
`/dashboard`	`dashboard/page.tsx`	Dashboard home
`/dashboard/new`	`dashboard/new/page.tsx`	Job creation form
`/dashboard/jobs`	`dashboard/jobs/page.tsx`	Jobs history list
`/dashboard/jobs/{id}`	`dashboard/jobs/[id]/page.tsx`	Job detail view

Layout Hierarchy Diagram

```
src/app/layout.tsx (ROOT)
├── Fonts: Inter, JetBrains Mono
├── globals.css imported
├── <html lang="en" className="dark">
└── <PageTransition>
    │
    ├── src/app/page.tsx
    │   └── Landing Page (/)
    │
    ├── src/app/(auth)/layout.tsx
    │   └── <GoogleOAuthProvider>
    │       └── src/app/(auth)/login/page.tsx
    │           └── Login Page (/login)
    │
    └── src/app/(dashboard)/layout.tsx
        └── <AuthGuard>
            └── <AppShell>
                │
                ├── <TopBar />
                ├── <Sidebar />
                ├── <main>  ← Children render here
                └── <Inspector />
                    │
                    ├── src/app/(dashboard)/dashboard/page.tsx
                    │   └── Dashboard (/dashboard)
                    │
                    ├── src/app/(dashboard)/dashboard/new/page.tsx
                    │   └── New Job (/dashboard/new)
                    │
                    ├── src/app/(dashboard)/dashboard/jobs/page.tsx
                    │   └── Jobs List (/dashboard/jobs)
                    │
                    └── src/app/(dashboard)/dashboard/jobs/[id]/page.tsx
                        └── Job Detail (/dashboard/jobs/{id})
```

Navigation Components

Sidebar Navigation

File: `src/components/layout/Sidebar.tsx`

Navigation Items:

```
const navItems = [
    { label: 'Dashboard', href: '/dashboard', icon: <GridIcon /> },
    { label: 'New Job', href: '/dashboard/new', icon: <PlusIcon /> },
    { label: 'Jobs History', href: '/dashboard/jobs', icon: <ClockIcon /> },
];
```

Active State Logic:

```
const isActive = pathname === item.href ||
    (item.href !== '/dashboard' && pathname.startsWith(item.href));
```

Simple Explanation

The sidebar shows three links. The current page is highlighted. "Dashboard" is only highlighted when you're exactly on `/dashboard`, but "Jobs History" is highlighted for `/dashboard/jobs` AND `/dashboard/jobs/123`.

Technical Explanation

- `usePathname()` from Next.js gives current URL path
- Exact match for dashboard root (avoids highlighting for all dashboard routes)
- Prefix match for other routes (job detail pages highlight "Jobs History")
- Active indicator uses Framer Motion `layoutId` for smooth animation

TopBar Navigation

File: `src/components/layout/TopBar.tsx`

- **Logo:** Links to dashboard (visual only, no click handler)
- **Inspector Toggle:** Shows/hides right panel
- **User Avatar:** Shows user picture from Google
- **Logout Button:** Calls `logout()` and redirects to `/login`

Programmatic Navigation

Router Usage

All programmatic navigation uses Next.js `useRouter`:

```
import { useRouter } from 'next/navigation';

function Component() {
    const router = useRouter();

    // Navigate to new page
    router.push('/dashboard/jobs/123');

    // Replace current URL (no back button)
    router.replace('/login');
}
```

Navigation Patterns by Action

Action	Method	Destination
After login	`router.replace()`	`/dashboard`
After logout	`router.replace()`	`/login`
Create job success	`router.push()`	`/dashboard/jobs/{id}`
Click job card	`router.push()`	`/dashboard/jobs/{id}`
Back from job detail	`router.push()`	`/dashboard/jobs`
Click sidebar nav	`<Link>` component	Various

Page Transitions

File: `src/components/layout/PageTransition.tsx`

Wraps all pages in Framer Motion `AnimatePresence`:

```
export function PageTransition({ children }) {
    const pathname = usePathname();

    return (
        <AnimatePresence mode="wait">
            <motion.div
                key={pathname}
                variants={pageVariants}
                initial="initial"
                animate="animate"
                exit="exit"
                transition={pageTransition}
            >
                {children}
            </motion.div>
        </AnimatePresence>
    );
}
```

Animation Effect: - **Enter:** Fade in + slide up from y: 20 - **Exit:** Fade out + slide up to y: -20 - **Duration:** 300ms with easeOut curve

Dynamic Route Parameters

Job Detail Page: `[id]`

File: `src/app/(dashboard)/dashboard/jobs/[id]/page.tsx`

Next.js 15+ Pattern: Route params are now a Promise:

```
interface JobDetailPageProps {
    params: Promise<{ id: string }>;
}

export default function JobDetailPage({ params }: JobDetailPageProps) {
    const { id } = use(params);  // React.use() to unwrap Promise

    // Use id for API calls, WebSocket, etc.
}
```

Simple Explanation

The `[id]` folder name tells Next.js this is a dynamic segment. Whatever you put in the URL (like `abc123`) becomes available as `id`.

Technical Explanation

- `use()` is a React 19 hook for unwrapping Promises in render
- This pattern is required in Next.js 15+ App Router
- The `id` is used to fetch job data and connect to WebSocket

State Management

> **Confidence:** CONFIRMED | **Source:** Direct code analysis of stores/ and CONTRACTS.md

---

Overview

Seone uses **Zustand** for global state management. There are **three stores**, each with a single responsibility:

Store	Purpose	Persistence
`useAuthStore`	Authentication state (user, tokens)	sessionStorage
`useJobStore`	Active job state (job detail, clips, WS status)	None (reset on route change)
`useAppStore`	UI state (sidebar, inspector)	None (persists in memory)

Store Contracts

These are **normative invariants** from `CONTRACTS.md`. Violations are bugs.

useAuthStore Invariants

Property	Type	Guarantee
`user`	`User | null`	`null` means not authenticated
`isAuthenticated`	`boolean`	Always defined
`isLoading`	`boolean`	Always defined

useJobStore Invariants

Property	Type	Guarantee
`job`	`Job | null`	`null` is valid transitional state
`liveClips`	`Clip[]`	Always an array, never undefined
`wsConnected`	`boolean`	Always defined

Reset Semantics

- `reset()` is **synchronous**
- After `reset()`: `{ job: null, liveClips: [], wsConnected: false }`
- Components **will render** during reset → fetch gap
- Components **must not crash** when `job` is `null`

Store: `useAuthStore`

File: `src/stores/auth.ts`

State Shape

```
interface AuthStore extends AuthState {
    user: User | null;
    isAuthenticated: boolean;
    isLoading: boolean;

    setUser: (user: User | null) => void;
    setLoading: (isLoading: boolean) => void;
    initialize: () => Promise<void>;
    logout: () => Promise<void>;
}
```

Persistence

```
persist(
    (set, get) => ({ ... }),
    {
        name: 'seone-auth',
        storage: createJSONStorage(() => sessionStorage),
        partialize: (state) => ({
            user: state.user,
            isAuthenticated: state.isAuthenticated,
        }),
    }
)
```

What's Persisted: `user` and `isAuthenticated` — NOT `isLoading`

Storage: sessionStorage (cleared when browser closes)

Why Not localStorage? Security — session should end when user closes browser.

Actions

`setUser(user: User | null)`

Sets user and derives `isAuthenticated`:

```
set({
    user,
    isAuthenticated: !!user,
    isLoading: false,
});
```

`initialize()`

Called on app load to restore session:

```
const token = getAuthToken();
const currentState = get();

if (token && currentState.user) {
    // Token exists and we have persisted user → authenticated
    set({ isLoading: false, isAuthenticated: true });
} else if (!token) {
    // No token → not authenticated
    set({ user: null, isAuthenticated: false, isLoading: false });
} else {
    // Token but no user → inconsistent, clear state
    set({ user: null, isAuthenticated: false, isLoading: false });
}
```

Key Insight: Does NOT call `/me` API. Relies on persisted state + token presence.

`logout()`

```
set({ isLoading: true });
await logoutService();  // Calls POST /api/v1/auth/logout
set({ user: null, isAuthenticated: false, isLoading: false });
```

Store: `useJobStore`

File: `src/stores/job.ts`

State Shape

```
interface JobState {
    job: Job | null;
    liveClips: Clip[];
    wsConnected: boolean;
    lastEventAt: string | null;
    isLoading: boolean;
    error: string | null;

    setJob: (job: Job) => void;
    updateJob: (updates: Partial<Job>) => void;
    addClip: (clip: Clip) => void;
    setWsConnected: (connected: boolean) => void;
    setLastEventAt: (timestamp: string) => void;
    setError: (error: string | null) => void;
    setLoading: (isLoading: boolean) => void;
    reset: () => void;
}
```

Clip Management

Clip Deduplication Key

```
const getClipKey = (clip: Clip): string => {
    const index = Number(clip.index);
    if (Number.isFinite(index)) return `index:${index}`;
    if (clip.url) return `url:${clip.url}`;
    if (clip.filename) return `file:${clip.filename}`;
    return `unknown:${JSON.stringify(clip)}`;
};
```

Priority: index → url → filename → stringified object

Clip Sorting

```
const sortClips = (clips: Clip[]): Clip[] => {
    return [...clips].sort((a, b) => {
        const aValid = Number.isFinite(Number(a.index));
        const bValid = Number.isFinite(Number(b.index));

        if (aValid && bValid) return aIndex - bIndex;
        if (aValid) return -1;
        if (bValid) return 1;
        return 0;
    });
};
```

Clips with valid indices come first, sorted ascending.

Clip Merging

```
const mergeClips = (base: Clip[], incoming: Clip[]): Clip[] => {
    if (base.length === 0) return sortClips(incoming);
    if (incoming.length === 0) return sortClips(base);

    const byKey = new Map<string, Clip>();
    for (const clip of base) byKey.set(getClipKey(clip), clip);
    for (const clip of incoming) byKey.set(getClipKey(clip), clip);

    return sortClips(Array.from(byKey.values()));
};
```

Incoming clips overwrite existing clips with same key.

Actions

`setJob(job: Job)`

```
set((state) => {
    const isSameJob = state.job?.id === job.id;
    const mergedClips = mergeClips(
        isSameJob ? state.liveClips : [],
        job.output?.clips ?? []
    );

    return { job, liveClips: mergedClips, error: null };
});
```

If navigating to same job: Merge existing liveClips with new clips **If navigating to different job:** Start fresh with

job's clips

`updateJob(updates: Partial<Job>)`

Shallow merge for incremental WebSocket updates:

```
set((state) => ({
    job: state.job ? { ...state.job, ...updates } : null
}));
```

`addClip(clip: Clip)`

Add single clip from WebSocket event:

```
set((state) => {
    const clipKey = getClipKey(clip);
    if (state.liveClips.some(existing => getClipKey(existing) === clipKey)) {
        return {};  // Already have this clip, no-op
    }
    return { liveClips: sortClips([...state.liveClips, clip]) };
});
```

`reset()`

Clear all state for job navigation:

```
set({
    job: null,
    liveClips: [],
    wsConnected: false,
    lastEventAt: null,
    isLoading: false,
    error: null
});
```

Store: `useAppStore`

File: `src/stores/app.ts`

State Shape

```
interface AppState {
    isInspectorOpen: boolean;
    toggleInspector: () => void;
    setInspectorOpen: (open: boolean) => void;

    isSidebarOpen: boolean;  // Mobile only
    toggleSidebar: () => void;
    setSidebarOpen: (open: boolean) => void;

    activeJobId: string | null;
    setActiveJobId: (id: string | null) => void;
}
```

Simple Explanation

This store tracks which panels are open/closed and which job is currently selected (for the Inspector panel).

Default Values

```
{
    isInspectorOpen: true,   // Inspector open by default
    isSidebarOpen: false,    // Mobile sidebar closed by default
    activeJobId: null,
}
```

Data Flow Diagram

```
┌──────────────────────────────────────────────────────────────┐
│                        USER ACTIONS                          │
├──────────────────────────────────────────────────────────────┤
│  Login │ Navigate │ Submit Job │ Select Job │ Toggle Panel    │
└────────┬──────────┬────────────┬────────────┬────────────────┘
         │          │            │            │
         ▼          │            │            │
┌────────────────┐  │            │            │
│ useAuthStore   │  │            │            │
│ ───────────────│  │            │            │
│ user           │  │            │            │
│ isAuthenticated│  │            │            │
│ isLoading      │  │            │            │
└────────────────┘  │            │            │
                    │            │            │
                    ▼            │            │
        ┌──────────────────┐     │            │
        │ Route Navigation │     │            │
        │ AuthGuard check  │     │            │
        └──────────────────┘     │            │
                                 │            │
                                 ▼            │
                  ┌──────────────────────┐    │
                  │ useJobStore          │    │
                  │ ─────────────────────│    │
                  │ job (from REST)      │    │
                  │ liveClips (from WS)  │    │
                  │ wsConnected          │    │
                  │ error                │    │
                  └──────────────────────┘    │
                           ▲                  │
                           │                  │
            ┌──────────────┴──────────────┐   │
            │                             │   │
   ┌────────────────┐   ┌────────────────┐   │
   │   REST API     │   │   WebSocket    │   │
   │ GET /jobs/{id} │   │ WS /ws/jobs/{id}│  │
   └────────────────┘   └────────────────┘   │
                                             │
                                             ▼
                           ┌──────────────────────┐
                           │ useAppStore          │
                           │ ─────────────────────│
                           │ isInspectorOpen      │
                           │ isSidebarOpen        │
                           │ activeJobId          │
                           └──────────────────────┘
```

Component Usage Patterns

Accessing Store State

```
// ✅ Correct: Select only needed state (minimal re-renders)
const user = useAuthStore(state => state.user);
const job = useJobStore(state => state.job);

// ❌ Avoid: Selecting entire store (re-renders on any change)
const authStore = useAuthStore();  // Don't do this
```

Accessing Store Actions

```
// Actions can be selected individually
const setJob = useJobStore(state => state.setJob);
const reset = useJobStore(state => state.reset);

// Or destructured together
const { job, setJob, updateJob, reset } = useJobStore();
```

Component Guards

Required guard patterns for null safety:

```
// PipelineTimeline
const job = useJobStore(state => state.job);
if (!job) return null;  // Required guard

// ClipGallery
const liveClips = useJobStore(state => state.liveClips);
const clips = liveClips.length > 0 ? liveClips : (job?.output?.clips || []);
if (clips.length === 0) return null;  // Required guard
```

API and Backend Integration

> **Confidence:** CONFIRMED | **Source:** Direct code analysis of services/auth.ts and hooks

---

Overview

The frontend communicates with the backend via:

1. **REST API** — For CRUD operations (auth, jobs, pages)
2. **WebSocket** — For real-time job updates (covered in `06_real_time_updates/`)

All REST requests use the `authFetch()` wrapper which: - Attaches JWT token from cookie - Handles 401 responses (session expiry) - Redirects to login on auth failure

---

Base Configuration

File: `src/lib/config.ts`

```
export const config = {
    api: {
        baseUrl: process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8000',
        version: 'v1',
        timeout: 30000,
    },
    ws: {
        baseUrl: process.env.NEXT_PUBLIC_WS_URL || 'ws://localhost:8000',
        reconnectAttempts: 5,
        reconnectDelay: 1000,
    },
    media: {
        dataBaseUrl: process.env.NEXT_PUBLIC_DATA_URL || 'http://localhost:8000/data',
    },
};
```

Environment Variables

Variable	Required	Default	Purpose
`NEXT_PUBLIC_API_URL`	Yes (prod)	`http://localhost:8000`	REST API base URL
`NEXT_PUBLIC_WS_URL`	Yes (prod)	`ws://localhost:8000`	WebSocket base URL
`NEXT_PUBLIC_DATA_URL`	Yes (prod)	`http://localhost:8000/data`	Media files URL
`NEXT_PUBLIC_GOOGLE_CLIENT_ID`	Yes	`""`	Google OAuth client ID

Endpoint Configuration

File: `src/lib/config.ts`

```
export const endpoints = {
    auth: {
        google: '/api/v1/auth/google',
        me: '/api/v1/auth/me',
        logout: '/api/v1/auth/logout',
    },
    jobs: {
        list: '/api/v1/jobs',
        create: '/api/v1/jobs',
        get: (id: string) => `/api/v1/jobs/${id}`,
        delete: (id: string) => `/api/v1/jobs/${id}`,
    },
    pages: {
        list: '/api/v1/pages',
        get: (id: string) => `/api/v1/pages/${id}`,
    },
    ws: {
        job: (jobId: string) => `/ws/jobs/${jobId}`,
    },
};
```

---

authFetch Wrapper

File: `src/services/auth.ts`

```
export async function authFetch(
    endpoint: string,
    options: RequestInit = {}
): Promise<Response> {
    const token = getAuthToken();

    const headers = new Headers(options.headers);
    if (token) {
        headers.set('Authorization', `Bearer ${token}`);
    }

    const response = await fetch(getApiUrl(endpoint), {
        ...options,
        headers,
    });

    // Handle token expiry
    if (response.status === 401) {
        clearAuthToken();
        if (typeof window !== 'undefined') {
            alert('Session expired. Please log in again.');
        }
        window.location.href = '/login';
        throw new Error('Session expired');
    }

    return response;
}
```

Simple Explanation

Every API call goes through this wrapper. It automatically adds your login token and handles session expiry by redirecting you to the login page.

Technical Explanation

- Reads JWT from cookie (`seone_token`)
- Attaches as `Authorization: Bearer <token>` header
- On 401 response: clears cookie, shows alert, redirects to `/login`, throws error
- The thrown error prevents further execution during redirect

Why This Matters in Production

Without this wrapper, a single component forgetting to add the token would cause auth failures. Centralized handling ensures consistency.

API Contracts by Endpoint

1. Authentication

POST /api/v1/auth/google

Purpose: Exchange Google ID token for Seone JWT

Request:

```
POST /api/v1/auth/google?token={google_id_token}
```

Response (Success):

```
{
    "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "token_type": "bearer"
}
```

Frontend Handling:

```
const response = await fetch(url.toString(), { method: 'POST' });
const data = await response.json();
setAuthToken(data.access_token, 1800);  // Store in cookie
```

Error Handling: - Non-200: Parse JSON for `detail` or `message` field - Display error to user on login page

---

GET /api/v1/auth/me

Purpose: Fetch current user info (optional — not used in current implementation)

Request:

```
GET /api/v1/auth/me
Authorization: Bearer {token}
```

Response:

```
{
    "id": "user-uuid",
    "email": "user@creativefuel.io",
    "name": "John Doe",
    "picture": "https://...",
    "role": "user"
}
```

Note: Currently the frontend does NOT call this endpoint. User info is extracted from Google ID token payload instead.

POST /api/v1/auth/logout

Purpose: Invalidate session on server (optional cleanup)

Request:

```
POST /api/v1/auth/logout
Authorization: Bearer {token}
```

Frontend Handling:

```
try {
    await fetch(getApiUrl(endpoints.auth.logout), {
        method: 'POST',
        headers: { 'Authorization': `Bearer ${token}` },
    });
} catch {
    // Ignore logout errors — clear token regardless
}
clearAuthToken();
```

---

2. Jobs

GET /api/v1/jobs

Purpose: List jobs for current user with pagination

Request:

```
GET /api/v1/jobs?page=1&per_page=10
Authorization: Bearer {token}
```

Response:

```
{
    "items": [
        {
            "id": "job-uuid",
            "status": "completed",
            "phase": "completed",
            "progress": 100,
            "clip_count": 3,
            "created_at": "2026-01-19T10:00:00Z",
            "completed_at": "2026-01-19T10:05:00Z"
        }
    ],
    "total": 42,
    "page": 1,
    "page_size": 10,
    "has_more": true
}
```

Frontend Handling: - Normalizes various response formats (array, wrapped object) - Handles missing pagination fields with defaults - Updates `useJobs` hook state

---

POST /api/v1/jobs

Purpose: Create a new job

Request:

```
POST /api/v1/jobs
Authorization: Bearer {token}
Content-Type: application/json

{
    "url": "https://youtube.com/watch?v=abc123",
    "min_duration": 1.0,
    "max_duration": 5.0,
    "count": 3,
    "pages": ["Modern Minimal", "Bold Creator"],
    "copy_mode": "en",
    "language": null,
    "extra_config": {
        "mode": "ai",
        "ui_language_selection": "en"
    }
}
```

Field Mapping: | Frontend Form | API Field | Transformation | |---------------|-----------|----------------| | `youtubeUrl` | `url` | Direct | | `minDuration` (seconds) | `min_duration` (minutes) | Divide by 60, clamp to [0.5, 10] | | `maxDuration` (seconds) | `max_duration` (minutes) | Divide by 60, clamp to [0.5, 10] | | `clipCount` | `count` | Direct | | `selectedPages` (IDs) | `pages` (names) | Map ID → page.name | | `language` | `copy_mode` | 'auto' → 'en', else direct | | `copyMode` | `extra_config.mode` | Stored for reference |

Response (Success):

```
{
    "id": "new-job-uuid",
    "status": "queued",
    "ws_url": "/ws/jobs/new-job-uuid",
    "message": "Job created successfully"
}
```

Frontend Handling: - On success: Navigate to `/dashboard/jobs/{id}` - On error: Display error message in form

---

GET /api/v1/jobs/{id}

Purpose: Fetch single job details

Request:

```
GET /api/v1/jobs/{id}
Authorization: Bearer {token}
```

Response:

```
{
    "id": "job-uuid",
    "status": "completed",
    "phase": "completed",
    "fork_join": {
        "fork_entered_at": "2026-01-19T10:01:00Z",
        "join_satisfied_at": "2026-01-19T10:04:00Z",
        "is_forked": true,
        "join_satisfied": true
    },
    "steps": {
        "download": { "status": "completed" },
        "transcribe": { "status": "completed" },
        "analyze": { "status": "completed" },
        "smart_render": { "status": "completed" }
    },
    "progress": 100,
    "current_step": "smart_render",
    "clip_count": 3,
    "created_at": "2026-01-19T10:00:00Z",
    "started_at": "2026-01-19T10:00:05Z",
    "completed_at": "2026-01-19T10:05:00Z",
    "output": {
        "clips": [
            { "index": 0, "url": "/clips/job-uuid/clip_0.mp4", "filename": "clip_0.mp4" },
            { "index": 1, "url": "/clips/job-uuid/clip_1.mp4", "filename": "clip_1.mp4" },
            { "index": 2, "url": "/clips/job-uuid/clip_2.mp4", "filename": "clip_2.mp4" }
        ]
    }
}
```

Frontend Handling: - On success: Call `setJob(data)` to update store - On 403: Display "This job does not belong to you" - On 404: Display "Job not found" - On error: Display generic error

---

3. Pages (Templates)

GET /api/v1/pages

Purpose: List available templates

Request:

```
GET /api/v1/pages
Authorization: Bearer {token}
```

Response:

```
{
    "pages": [
        {
            "id": "page-uuid",
            "name": "Modern Minimal",
            "slug": "modern-minimal",
            "category": "Trending",
            "description": "Clean, minimal design",
            "thumbnailUrl": "/thumbnails/modern-minimal.jpg"
        }
    ]
}
```

Frontend Handling: - Cached in-memory for session (module-level variable) - Falls back to mock data on error (development only)

Error Handling Patterns

Standard Error Response

```
{
    "detail": "Human-readable error message"
}
```

Error Parsing

```
if (!response.ok) {
    const error = await response.json().catch(() => ({
        detail: 'Request failed'
    }));
    throw new Error(error.detail || error.message || 'Unknown error');
}
```

Status Code Handling

Status	Meaning	Frontend Action
200	Success	Process response
201	Created	Process response
401	Unauthorized	Clear token, redirect to login
403	Forbidden	Show "access denied" error
404	Not Found	Show "not found" error
422	Validation Error	Show field-specific errors
500	Server Error	Show generic error, allow retry

Real-Time Updates

> **Confidence:** CONFIRMED | **Source:** Direct code analysis of useJobWebSocket.ts

---

Overview

Seone uses **WebSocket + REST polling** for real-time job updates:

Mechanism	Purpose	Frequency
WebSocket	Real-time events (step changes, clips)	Event-driven
REST Polling	State reconciliation	Every 3 seconds
REST Fetch	Initial load, final reconciliation	On mount, on job_completed

This hybrid approach ensures the frontend always converges to the correct state, even if WebSocket events are missed.

WebSocket Architecture

File: `src/hooks/useJobWebSocket.ts`

Connection URL

```
const wsUrl = `${getWsUrl(endpoints.ws.job(jobId))}?token=${encodeURIComponent(token)}`;
// Example: ws://localhost:8000/ws/jobs/abc123?token=eyJ...
```

Protocol: `ws://` (development) or `wss://` (production) **Authentication:** JWT passed as query parameter `?token=`

Connection Lifecycle

```
1. Component Mounts → useJobWebSocket(jobId) called
2. Check auth token → If expired, show error, don't connect
3. Create WebSocket → new WebSocket(wsUrl)
4. onopen → Set connected, reset reconnect counter, fetch REST snapshot
5. onmessage → Parse event, update store
6. onclose → Handle reconnection
7. onerror → Log warning, close socket
8. Component Unmounts → Close socket, clear timeouts
```

Event Types

`connected`

When: Immediately after WebSocket opens **Payload:** `{ message: "Connected to job {id}" }` **Action:** Log to console (no state change)

`step_started`

When: Worker begins a processing step **Payload:** `{ step: "download" | "transcribe" | "analyze" | "smart_render" }` **Action:** Update `job.current_step` and `job.status`

```
case 'step_started':
    const step = data.payload.step;
    let status;
    switch (step) {
        case 'smart_render': status = 'rendering'; break;
        case 'transcribe': status = 'transcribing'; break;
        case 'analyze': status = 'analyzing'; break;
        case 'download': status = 'downloading'; break;
    }
    updateJob({ current_step: step, status });
```

`step_completed`

When: Worker finishes a processing step **Payload:** `{ step: "download" }` **Action:** Currently no-op (could be used for visual feedback)

`clip_ready`

When: A single clip is rendered **Payload:**

```
{
    "clip_index": 0,
    "clip_url": "/clips/job-id/clip_0.mp4",
    "clips_ready": 1,
    "clip_count": 3
}
```

Action: Add clip to `liveClips`, update progress

```
case 'clip_ready':
    const clip = {
        index: data.payload.clip_index,
        url: data.payload.clip_url,
        filename: data.payload.clip_url.split('/').pop()
    };
    addClip(clip);
    updateJob({
        progress: (data.payload.clips_ready / data.payload.clip_count) * 100
    });
```

`job_completed`

When: All clips are rendered **Payload:**

```
{
    "output": {
        "clips": [
            { "index": 0, "url": "...", "filename": "..." },
            ...
        ]
    }
}
```

Action: Update status to completed, set progress to 100, fetch REST for final reconciliation

```
case 'job_completed':
    updateJob({
        status: 'completed',
        progress: 100,
        completed_at: new Date().toISOString(),
        output: data.payload.output
    });
    fetchJob();  // REST reconciliation
```

`job_failed`

When: Job processing fails **Payload:** `{ error: "Error message" }` **Action:** Update status to failed, store error message, close socket

```
case 'job_failed':
    updateJob({
        status: 'failed',
        error_message: data.payload.error
    });
    ws.close();
```

Authentication Handling

Token Validation Before Connect

```
const token = getValidAuthToken(60);  // 60 second buffer
if (!token) {
    setError('Session expired. Please refresh the page or log in again.');
    setWsConnected(false);
    return;
}
```

Why Buffer? If token expires during connection, the WebSocket will be rejected. The 60-second buffer ensures we don't attempt connections with nearly-expired tokens.

Auth-Related Close Codes

```
const AUTH_CLOSE_CODES = new Set([
    4001,  // Custom: Unauthorized
    4003,  // Custom: Forbidden
    1008,  // Policy Violation (standard)
]);
```

When WebSocket closes with these codes: 1. Set `needsAuthRef.current = true` 2. Check if token is still valid 3. If expired → Show error, don't reconnect 4. If valid → Try reconnect (backend might have temporary issue)

Reconnection Logic

```
ws.onclose = (event) => {
    setWsConnected(false);

    // Don't reconnect on normal closure
    if (event.code === 1000) return;

    // Don't reconnect after max attempts
    if (reconnectAttemptsRef.current >= 5) {
        setError('Connection lost. Please refresh the page.');
        return;
    }

    // Exponential backoff: 1s, 2s, 4s, 8s, 10s (capped)
    const timeout = Math.min(1000 * Math.pow(2, reconnectAttemptsRef.current), 10000);
    reconnectAttemptsRef.current++;

    setTimeout(connect, timeout);
};
```

Simple Explanation

If the connection drops unexpectedly, we try again. Each retry waits longer (1 second, then 2, then 4...) to avoid hammering the server. After 5 tries, we give up and tell the user to refresh.

Technical Explanation

- `reconnectAttemptsRef` tracks retry count
- Exponential backoff with 10s cap
- Refs used instead of state to avoid dependency issues
- `mountedRef` prevents state updates after unmount

REST Polling

Why Poll?

WebSocket events can be missed: - Late connection (job started before WS connected) - Network interruption - Server restart - Browser tab throttling

Polling ensures eventual consistency.

Polling Implementation

```
useEffect(() => {
    if (!jobId || !mountedRef.current) return;

    // Stop polling for terminal jobs
    const isTerminal = job?.status === 'completed' ||
                       job?.status === 'failed' ||
                       job?.phase === 'completed' ||
                       job?.phase === 'failed';
    if (isTerminal) return;

    const intervalId = setInterval(() => {
        if (mountedRef.current) {
            fetchJob();
        }
    }, 3000);

    return () => clearInterval(intervalId);
}, [jobId, job?.status, job?.phase, fetchJob]);
```

Polling Rules

1. Poll every 3 seconds while job is active
2. Stop polling when job reaches terminal state
3. Polling continues even if WebSocket is connected (redundancy)
4. Each poll calls `setJob()` which merges clips

State Precedence

```
REST  → Initial snapshot (authoritative on mount)
WS    → Transitions (authoritative during session)
REST  → Final reconciliation on job_completed
```

Last-Write-Wins

```
// No timestamp comparison — last write wins
updateJob(updates);  // Just shallow merge
```

Why? Simplicity. Both REST and WS are authoritative sources. Conflicts are rare and resolve quickly due to polling.

Known Race Windows

Race	Impact	Mitigation
WS before REST completes	WS applied to stale state	fetchStatus gates WS connection
REST returns after WS update	REST overwrites	Acceptable; final state is correct
Clip arrives before job data	Clip stored in liveClips	Rendered when job data arrives

---

Conditional Connection

The WebSocket only connects for active jobs:

```
const shouldConnect =
    fetchStatus === 'success' &&
    job &&
    job.status !== 'completed' &&
    job.status !== 'failed';

useJobWebSocket(shouldConnect ? id : '');
```

Why? - No point connecting for completed jobs - Failed jobs won't send more events - Saves server resources

---

Data Flow Diagram

```
┌──────────────────────────────────────────────────────────┐
│                    JOB DETAIL PAGE MOUNT                 │
└──────────────────────────────────────────────────────────┘
                           │
                           ▼
                  ┌──────────────────┐
                  │ reset() store    │
                  │ fetchJob() REST  │
                  └──────────────────┘
                           │
                  ┌────────┴─────────┐
                  │                  │
                  ▼                  ▼
            ┌──────────────┐   ┌──────────────────┐
            │ setJob(data) │   │ setFetchStatus   │
            │ Merge clips  │   │ ('success')      │
            └──────────────┘   └──────────────────┘
                  │                  │
                  └────────┬─────────┘
                           │
                           ▼
                  ┌──────────────────┐
                  │ Is job terminal? │
                  │ (completed/failed)│
                  └──────────────────┘
                           │
                  ┌────────┴─────────┐
             ┌────────────────────────┐
             │ NO                     │ YES
             ▼                        ▼
    ────────────────          ────────────────
```

```
    ┌─────────────────┐             ┌─────────────────┐
    │ Connect WebSocket│             │ Skip WebSocket  │
    │ Start polling   │             │ Skip polling    │
    └─────────────────┘             └─────────────────┘
              │
              ▼
    ┌──────────────────────────────────────────┐
    │          REAL-TIME UPDATE LOOP           │
    ├──────────────────────────────────────────┤
    │                                          │
    │  ┌──────────────┐    ┌─────────────────┐ │
    │  │ WS Events    │    │ REST Polling    │ │
    │  │ (immediate)  │    │ (every 3s)      │ │
    │  └──────┬───────┘    └────────┬────────┘ │
    │         │                     │          │
    │         │    ┌────────────────┘          │
    │         │    │                           │
    │         ▼    ▼                           │
    │    ┌──────────────────┐                  │
    │    │ updateJob()      │                  │
    │    │ addClip()        │                  │
    │    │ setJob()         │                  │
    │    └──────────────────┘                  │
    │                                          │
    └──────────────────────────────────────────┘
              │
              │ job_completed event
              ▼
    ┌─────────────────┐
    │ Final REST fetch│
    │ Stop polling    │
    │ Close WebSocket │
    └─────────────────┘
```

Protocol Validation

File: `src/lib/config.ts`

On module load, validates WebSocket protocol matches page security:

```
function validateWsConfig(): void {
    if (typeof window === 'undefined') return;

    const isSecurePage = window.location.protocol === 'https:';
    const isSecureWs = config.ws.baseUrl.startsWith('wss://');

    if (isSecurePage && !isSecureWs) {
        console.error(
            '[FATAL CONFIG] Secure page (https) attempting non-secure WebSocket (ws://).'
        );
    }
}
```

Why? HTTPS pages cannot connect to ws:// (mixed content). This catches misconfiguration at startup.

Authentication and Sessions

> **Confidence:** CONFIRMED | **Source:** Direct code analysis of services/auth.ts, stores/auth.ts, login page

---

Overview

Seone uses **Google OAuth 2.0** for authentication:

1. User clicks "Sign in with Google"
2. Google ID token received via `@react-oauth/google`
3. Token exchanged with backend for Seone JWT
4. JWT stored in cookie, user info in sessionStorage
5. JWT attached to all API requests

Authentication Flow

```
┌─────────────────────────────────────────────────────────┐
│                    USER CLICKS LOGIN                    │
└─────────────────────────────────────────────────────────┘
                          │
                          ▼
                  ┌─────────────────┐
                  │ GoogleLogin     │
                  │ Component       │
                  │ (@react-oauth)  │
                  └─────────────────┘
                          │
                          │ Google OAuth popup
                          │ User selects account
                          ▼
                  ┌─────────────────┐
                  │ onSuccess       │
                  │ callback        │
                  │ credential: JWT │
                  └─────────────────┘
                          │
                          ▼
                  ┌─────────────────┐
                  │ Decode Google   │
                  │ ID Token        │
                  │ (client-side)   │
                  └─────────────────┘
                          │
                          ▼
                  ┌─────────────────┐
                  │ Check Domain    │
                  │ @creativefuel.io │
                  │ required        │
                  └─────────────────┘
                          │
```

```
            ┌──────────────┬──────────────┐
            │ ALLOWED                     │ BLOCKED
            ▼                             ▼
┌──────────────────┐         ┌──────────────────┐
│ POST /api/v1/auth/  │         │ Show error:         │
│ google?token=...    │         │ "Only @xyz allowed" │
└──────────────────┘         └──────────────────┘
            │
            ▼
┌──────────────────┐
│ Receive Seone JWT   │
│ access_token        │
└──────────────────┘
            │
            ▼
┌──────────────────┐
│ Store JWT in cookie │
│ (seone_token)       │
│ httpOnly=false      │
│ secure=true (prod)  │
│ sameSite=strict     │
└──────────────────┘
            │
            ▼
┌──────────────────┐
│ Extract user info   │
│ from Google token   │
│ (sub, email, name,  │
│  picture)           │
└──────────────────┘
            │
            ▼
┌──────────────────┐
│ setUser() in store  │
│ Persisted to        │
│ sessionStorage      │
└──────────────────┘
            │
            ▼
┌──────────────────┐
│ router.replace()    │
│ → /dashboard        │
└──────────────────┘
```

Domain Restriction

File: `src/lib/config.ts`

```
auth: {
    allowedDomain: ['creativefuel.io'],
}
```

File: `src/services/auth.ts`

```
export function isAllowedDomain(email: string): boolean {
    const domain = email.split('@')[1]?.toLowerCase();
    return (config.auth.allowedDomain as readonly string[]).includes(domain);
}
```

Login Page Check

```
const payload = JSON.parse(atob(response.credential.split('.')[1]));

if (!isAllowedDomain(payload.email)) {
    setError(`Only @${config.auth.allowedDomain.join(', @')} accounts are allowed.`);
    return;
}
```

Why Client-Side? Fail fast. Don't waste a backend round-trip for unauthorized domains.

Backend Also Checks: The backend performs its own domain validation. Client-side is UX optimization only.

JWT Token Management

Storing the Token

File: `src/services/auth.ts`

```
export function setAuthToken(token: string, expiresInSeconds?: number): void {
    const expires = expiresInSeconds
        ? expiresInSeconds / 86400  // Convert seconds to days
        : config.auth.tokenExpiry;  // Default: 7 days

    Cookies.set(config.auth.tokenCookieName, token, {
        expires,
        secure: process.env.NODE_ENV === 'production',
        sameSite: 'strict',
    });
}
```

Cookie Name: `seone_token` **Expiry:** 30 minutes (backend default), frontend stores 7 days as fallback

Reading the Token

```
export function getAuthToken(): string | undefined {
    return Cookies.get(config.auth.tokenCookieName);
}
```

Clearing the Token

```
export function clearAuthToken(): void {
    Cookies.remove(config.auth.tokenCookieName);
}
```

Token Expiry Handling

JWT Payload Decoding

```
export function decodeJwtPayload(token: string): JwtPayload | null {
    try {
        const parts = token.split('.');
        if (parts.length !== 3) return null;

        const payload = parts[1];
        const decoded = atob(payload.replace(/-/g, '+').replace(/_/g, '/'));
        return JSON.parse(decoded);
    } catch {
        return null;
    }
}
```

Note: This is NOT signature verification. Backend still validates the token. Client-side decoding is for expiry checks only.

Expiry Check

```
export function isTokenExpired(token: string, bufferSeconds: number = 60): boolean {
    const expiry = getTokenExpiry(token);
    if (!expiry) return true;  // Treat unparseable as expired

    const now = new Date();
    const bufferMs = bufferSeconds * 1000;
    return expiry.getTime() - bufferMs <= now.getTime();
}
```

Buffer: Default 60 seconds before actual expiry. Prevents edge case where token expires mid-request.

Get Valid Token

```
export function getValidAuthToken(bufferSeconds: number = 60): string | undefined {
    const token = getAuthToken();
    if (!token) return undefined;
    if (isTokenExpired(token, bufferSeconds)) return undefined;
    return token;
}
```

Used by WebSocket connection to ensure token won't expire during connection.

Session Persistence

Auth Store with Persistence

File: `src/stores/auth.ts`

```
export const useAuthStore = create<AuthStore>()(
    persist(
        (set, get) => ({ ... }),
        {
            name: 'seone-auth',
            storage: createJSONStorage(() => sessionStorage),
            partialize: (state) => ({
                user: state.user,
                isAuthenticated: state.isAuthenticated,
            }),
        }
    )
);
```

What's Persisted: - `user` (object with name, email, picture, etc.) - `isAuthenticated` (boolean)

What's NOT Persisted: - `isLoading` (always starts as true)

Session Initialization

```
initialize: async () => {
    const token = getAuthToken();
    const currentState = get();

    // Token + persisted user → authenticated
    if (token && currentState.user) {
        set({ isLoading: false, isAuthenticated: true });
        return;
    }

    // No token → not authenticated
    if (!token) {
        set({ user: null, isAuthenticated: false, isLoading: false });
        return;
    }

    // Token but no user → inconsistent, clear state
    set({ user: null, isAuthenticated: false, isLoading: false });
}
```

Key Decision: Does NOT call `/me` API. Relies on persisted state.

Why? 1. Faster page loads (no API wait) 2. User info from Google token is sufficient 3. If token is invalid, backend will return 401 which triggers login

---

AuthGuard Component

File: `src/components/layout/AuthGuard.tsx`

Purpose

- Protect routes that require authentication
- Redirect authenticated users away from login page

Implementation

```
export function AuthGuard({
    children,
    requireAuth = true,
    redirectTo,
}: AuthGuardProps) {
    const router = useRouter();
    const { isAuthenticated, isLoading, initialize } = useAuthStore();

    // Initialize auth on mount
    useEffect(() => {
        initialize();
    }, [initialize]);

    // Handle redirects
    useEffect(() => {
        if (isLoading) return;

        if (requireAuth && !isAuthenticated) {
            router.replace(redirectTo || '/login');
        } else if (!requireAuth && isAuthenticated) {
            router.replace(redirectTo || '/dashboard');
        }
    }, [isAuthenticated, isLoading, requireAuth, redirectTo, router]);

    // Loading state
    if (isLoading) {
        return <div className="auth-loading"><div className="auth-loading-spinner" /></div>;
    }

    // Don't render children if redirect needed
    if (requireAuth && !isAuthenticated) return null;
    if (!requireAuth && isAuthenticated) return null;

    return <>{children}</>;
}
```

Usage

```
// Dashboard layout - require auth
<AuthGuard requireAuth={true} redirectTo="/login">
    <AppShell>{children}</AppShell>
</AuthGuard>

// Login page - redirect if already authenticated
<AuthGuard requireAuth={false} redirectTo="/dashboard">
    <LoginPage />
</AuthGuard>
```

Logout Flow

TopBar Logout Button

```
const handleLogout = async () => {
    await logout();
    router.replace('/login');
};
```

Store Logout Action

```
logout: async () => {
    set({ isLoading: true });
    await logoutService();  // Calls POST /api/v1/auth/logout
    set({ user: null, isAuthenticated: false, isLoading: false });
}
```

Service Logout

```
export async function logout(): Promise<void> {
    const token = getAuthToken();

    if (token) {
        try {
            await fetch(getApiUrl(endpoints.auth.logout), {
                method: 'POST',
                headers: { 'Authorization': `Bearer ${token}` },
            });
        } catch {
            // Ignore logout errors — clear token regardless
        }
    }

    clearAuthToken();
}
```

Key Behavior: Even if backend logout fails, clear the cookie. User should be logged out locally.

Session Handling on 401

File: `src/services/auth.ts`

```
if (response.status === 401) {
    clearAuthToken();
    if (typeof window !== 'undefined') {
        alert('Session expired. Please log in again.');
    }
    window.location.href = '/login';
    throw new Error('Session expired');
}
```

Simple Explanation

If any API call returns "unauthorized," we assume the session is expired, clear everything, and send the user to login.

Technical Explanation

- 401 could mean: expired token, revoked token, tampered token
- Alert provides user feedback (temporary UX until toast system exists)
- Hard redirect via `window.location.href` ensures full page reload
- Thrown error prevents further code execution during redirect

Why This Matters in Production

Without centralized 401 handling, components might continue executing with invalid state, causing confusing errors or data inconsistencies.

Security Considerations

Aspect	Implementation	Rationale
Token Storage	Cookie (not localStorage)	Automatic inclusion in requests
Cookie Secure Flag	`secure: true` in production	Prevents transmission over HTTP
Cookie SameSite	`strict`	CSRF protection
Session Scope	sessionStorage	Clears when browser closes
Domain Restriction	Client + server validation	Fail fast + defense in depth
Token Expiry	30 min backend, client checks	Short-lived tokens reduce risk

UI Components and Design System

> **Confidence:** CONFIRMED | **Source:** Direct code analysis of components/ui, globals.css

---

Design System Overview

Seone uses a **custom CSS design system** with: - CSS Custom Properties (variables) for theming - CSS Modules for component scoping - Framer Motion for animations - No Tailwind or CSS-in-JS

File: `src/app/globals.css`

Design Tokens

Colors

```
:root {
  /* Background Colors - Deep Dark Palette */
  --bg-primary: #0a0a0b;
  --bg-secondary: #111113;
  --bg-tertiary: #1a1a1d;
  --bg-elevated: #222225;
  --bg-hover: #2a2a2e;

  /* Text Colors - Crisp Hierarchy */
  --text-primary: #ffffff;
  --text-secondary: #a1a1a6;
  --text-muted: #6e6e73;
  --text-disabled: #48484a;

  /* Accent Colors - Vibrant Indigo */
  --accent-primary: #6366f1;
  --accent-primary-hover: #818cf8;
  --accent-primary-muted: rgba(99, 102, 241, 0.15);

  /* Semantic Colors */
  --color-success: #22c55e;
  --color-warning: #f59e0b;
  --color-error: #ef4444;
  --color-info: #3b82f6;
}
```

Spacing

```
:root {
  --space-1: 4px;
  --space-2: 8px;
  --space-3: 12px;
  --space-4: 16px;
  --space-5: 20px;
  --space-6: 24px;
  --space-8: 32px;
  --space-10: 40px;
  --space-12: 48px;
  --space-16: 64px;
}
```

Border Radius

```
:root {
  --radius-sm: 6px;
  --radius-md: 10px;
  --radius-lg: 14px;
  --radius-xl: 20px;
  --radius-full: 9999px;
}
```

Typography

```
:root {
  --font-sans: 'Inter', -apple-system, BlinkMacSystemFont, sans-serif;
  --font-mono: 'JetBrains Mono', 'Fira Code', monospace;

  --text-xs: 0.75rem;    /* 12px */
  --text-sm: 0.875rem;   /* 14px */
  --text-base: 1rem;     /* 16px */
  --text-lg: 1.125rem;   /* 18px */
  --text-xl: 1.25rem;    /* 20px */
  --text-2xl: 1.5rem;    /* 24px */
  --text-3xl: 1.875rem;  /* 30px */
  --text-4xl: 2.25rem;   /* 36px */
}
```

Transitions

```
:root {
  --transition-fast: 150ms cubic-bezier(0.4, 0, 0.2, 1);
  --transition-base: 200ms cubic-bezier(0.4, 0, 0.2, 1);
  --transition-slow: 300ms cubic-bezier(0.4, 0, 0.2, 1);
}
```

Layout

```
:root {
  --sidebar-width: 260px;
  --inspector-width: 340px;
  --topbar-height: 60px;
}
```

Base Components

Button

File: `src/components/ui/Button.tsx`

Props

Prop	Type	Default	Description
`variant`	`'primary' | 'secondary' | 'ghost' | 'danger'`	`'primary'`	Visual style
`size`	`'sm' | 'md' | 'lg'`	`'md'`	Size variant
`isLoading`	`boolean`	`false`	Shows spinner, disables button
`leftIcon`	`ReactNode`	—	Icon before text
`rightIcon`	`ReactNode`	—	Icon after text
`disabled`	`boolean`	`false`	Disabled state

Usage

```
// Primary button
<Button onClick={handleSubmit}>Submit</Button>

// Secondary with loading
<Button variant="secondary" isLoading={isSubmitting}>
    Processing...
</Button>

// Ghost button (minimal style)
<Button variant="ghost" size="sm">Cancel</Button>

// With icon
<Button leftIcon={<PlusIcon />}>New Job</Button>
```

Animation

Uses Framer Motion for hover/tap effects:

```
whileHover={{ y: -1, filter: 'brightness(1.1)' }}
whileTap={{ scale: 0.97 }}
```

Input

File: `src/components/ui/Input.tsx`

Props

Prop	Type	Default	Description
`label`	`string`	—	Label text above input
`error`	`string`	—	Error message below input
`hint`	`string`	—	Hint text (shown if no error)
`leftIcon`	`ReactNode`	—	Icon inside input, left side
`rightElement`	`ReactNode`	—	Element inside input, right side

Usage

```
<Input
    label="YouTube URL"
    placeholder="https://youtube.com/watch?v=..."
    value={url}
    onChange={(e) => setUrl(e.target.value)}
    error={errors.url}
    leftIcon={<YoutubeIcon />}
/>
```

DualRangeSlider

File: `src/components/ui/DualRangeSlider.tsx`

Props

Prop	Type	Default	Description
`min`	`number`	—	Minimum value
`max`	`number`	—	Maximum value
`minValue`	`number`	—	Current min selection
`maxValue`	`number`	—	Current max selection
`step`	`number`	`1`	Step increment
`label`	`string`	—	Label text
`formatValue`	`(value: number) => string`	—	Display formatter
`onChange`	`(min, max) => void`	—	Change handler

Usage

```
<DualRangeSlider
    label="Clip Duration Range"
    min={30}
    max={600}
    minValue={formData.minDuration}
    maxValue={formData.maxDuration}
    step={15}
    formatValue={formatDuration}  // "1:00" format
    onChange={(min, max) => {
        updateField('minDuration', min);
        updateField('maxDuration', max);
    }}
/>
```

Animation Library

File: `src/lib/animations.ts`

Transition Presets

```
export const transitions = {
    fast: { duration: 0.15, ease: [0.4, 0, 0.2, 1] },
    base: { duration: 0.2, ease: [0.4, 0, 0.2, 1] },
    slow: { duration: 0.3, ease: [0.4, 0, 0.2, 1] },
    spring: { type: 'spring', stiffness: 400, damping: 30 },
};
```

Page Transitions

```
export const pageVariants: Variants = {
    initial: { opacity: 0, y: 20 },
    animate: { opacity: 1, y: 0 },
    exit: { opacity: 0, y: -20 },
};
```

List Animations

```
export const staggerContainer: Variants = {
    initial: {},
    animate: {
        transition: {
            staggerChildren: 0.05,
            delayChildren: 0.1,
        },
    },
};

export const listItemVariants: Variants = {
    initial: { opacity: 0, y: 10 },
    animate: { opacity: 1, y: 0 },
    exit: { opacity: 0, y: -10 },
};
```

Usage in Components

```
<motion.div
    className={styles.grid}
    variants={staggerContainer}
    initial="initial"
    animate="animate"
>
    {items.map((item) => (
        <motion.div key={item.id} variants={listItemVariants}>
            {item.content}
        </motion.div>
    ))}
</motion.div>
```

Utility Classes

Global CSS Classes

```
/* Text Colors */
.text-primary { color: var(--text-primary); }
.text-secondary { color: var(--text-secondary); }
.text-muted { color: var(--text-muted); }
.text-accent { color: var(--accent-primary); }

/* Backgrounds */
.bg-primary { background: var(--bg-primary); }
.bg-secondary { background: var(--bg-secondary); }

/* Glass Effect */
.glass {
    background: var(--glass-bg);
    backdrop-filter: blur(12px);
    border: 1px solid var(--glass-border);
}

/* Gradient Text */
.gradient-text {
    background: linear-gradient(135deg, var(--accent-primary) 0%, #a855f7 100%);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
}

/* Animations */
.animate-fadeIn { animation: fadeIn var(--transition-base) ease-out; }
.animate-spin { animation: spin 1s linear infinite; }
```

cn() Helper

File: `src/lib/utils.ts`

```
import { type ClassValue, clsx } from 'clsx';

export function cn(...inputs: ClassValue[]): string {
    return clsx(inputs);
}
```

Usage:

```
<div className={cn(styles.card, isActive && styles.active, className)}>
```

Component Patterns

CSS Module Structure

Each component has a paired `.module.css` file:

```
Button.tsx
Button.module.css
```

```
import styles from './Button.module.css';

<button className={styles.button}>Click</button>
```

Conditional Classes

```
<div className={cn(
    styles.step,
    isActive && styles.active,
    isDone && styles.done,
    isError && styles.error
)}>
```

Dynamic Styles

```
<div
    className={styles.range}
    style={{
        left: `${minPercent}%`,
        width: `${maxPercent - minPercent}%`
    }}
/>
```

Error Handling and Edge Cases

> **Confidence:** CONFIRMED | **Source:** Direct code analysis

---

Error Handling Strategy

Seone uses a **multi-layer error handling** approach:

Layer	Responsibility	Example
Service Layer	API errors, token expiry	`authFetch()` handles 401
Hooks	Data fetching errors	`useJobs` sets error state
Stores	State errors	`useJobStore.error`
Components	Display errors	Error UI rendering

API Error Handling

authFetch Session Expiry

File: `src/services/auth.ts`

```
if (response.status === 401) {
    clearAuthToken();
    alert('Session expired. Please log in again.');
    window.location.href = '/login';
    throw new Error('Session expired');
}
```

Behavior: Any 401 response triggers logout and redirect. No component-level handling needed.

Standard Error Parsing

```
if (!response.ok) {
    const error = await response.json().catch(() => ({
        detail: 'Request failed'
    }));
    throw new Error(error.detail || error.message || 'Unknown error');
}
```

Pattern: Try to get `detail` field (FastAPI standard), fall back to `message`, then generic.

Hook-Level Error Handling

useJobs

```
const [state, setState] = useState({
    items: [],
    isLoading: true,
    error: null,  // string | null
});

try {
    const response = await authFetch(...);
    if (!response.ok) throw new Error('Failed to fetch jobs');
    // ...success path
} catch (err) {
    setState(prev => ({
        ...prev,
        isLoading: false,
        error: err instanceof Error ? err.message : 'Failed to fetch jobs',
    }));
}
```

useJobWebSocket

```
// Connection errors
ws.onerror = () => {
    console.warn('WebSocket connection failed');
    ws.close();
};

// Auth errors from close codes
if (AUTH_CLOSE_CODES.has(event.code)) {
    setError('Session expired. Please refresh the page or log in again.');
    return;
}

// Max reconnection attempts
if (reconnectAttemptsRef.current >= 5) {
    setError('Connection lost. Please refresh the page.');
}
```

Component Error Display

JobsList Error State

```
if (error) {
    return (
        <div className={styles.error}>
            <p>{error}</p>
            <Button onClick={() => window.location.reload()} variant="secondary">
                Retry
            </Button>
        </div>
    );
}
```

Job Detail Page Error States

```
// Ownership/access errors
if (fetchStatus === 'error' && fetchError) {
    return (
        <div className={styles.errorContainer}>
            <div className={styles.errorCard}>
                <h2 className={styles.errorCode}>{fetchError.code}</h2>
                <p className={styles.errorMessage}>{fetchError.message}</p>
                <Button onClick={() => router.push('/dashboard/jobs')} variant="secondary">
                    Back to Jobs
                </Button>
            </div>
        </div>
    );
}
```

HTTP Status Mapping

```
if (!response.ok) {
    const status = response.status;
    if (status === 403) {
        throw { code: 403, message: 'This job does not belong to you.' };
    } else if (status === 404) {
        throw { code: 404, message: 'Job not found.' };
    } else {
        throw { code: status, message: 'Failed to load job.' };
    }
}
```

Form Validation Errors

useJobSubmit Validation

```
const validate = useCallback(():boolean => {
    const newErrors: FormErrors = {};

    if (!formData.youtubeUrl.trim()) {
        newErrors.youtubeUrl = 'YouTube URL is required';
    } else if (!isValidYouTubeUrl(formData.youtubeUrl)) {
        newErrors.youtubeUrl = 'Please enter a valid YouTube URL';
    }

    if (formData.minDuration >= formData.maxDuration) {
        newErrors.duration = 'Min duration must be less than max duration';
    }

    if (formData.selectedPages.length === 0) {
        newErrors.selectedPages = 'Please select at least one template';
    }

    setErrors(newErrors);
    return Object.keys(newErrors).length === 0;
}, [formData]);
```

Field Error Display

```
<Input
    value={formData.youtubeUrl}
    error={errors.youtubeUrl}  // Shown below input
/>

{errors.duration && (
    <span className={styles.error}>{errors.duration}</span>
)}
```

Edge Cases

Empty States

Scenario	Component	Display
No jobs	JobsList	"No jobs found" + Create button
No clips yet	ClipGallery	Returns `null` (hidden)
No active job	Inspector	"Select a job to view details"
No templates	TemplateSelector	Falls back to mock data

```
// JobsList empty state
if (items.length === 0) {
    return (
        <div className={styles.empty}>
            <h3>No jobs found</h3>
            <p>Create your first job to get started</p>
            <Button onClick={() => router.push('/dashboard/new')}>
                Create Job
            </Button>
        </div>
    );
}
```

Loading States

```
// Initial loading with skeleton
if (isLoading && items.length === 0) {
    return (
        <div className={styles.grid}>
            {[1, 2, 3, 4, 5, 6].map((i) => (
                <div key={i} className={styles.skeleton} />
            ))}
        </div>
    );
}

// Job detail loading
if (fetchStatus === 'loading') {
    return (
        <div className={styles.loading}>
            <div className={styles.spinner} />
            <p>Loading job details...</p>
        </div>
    );
}
```

Null Guards

Required by CONTRACTS.md:

```
// PipelineTimeline
const job = useJobStore(state => state.job);
if (!job) return null;

// ClipGallery
const clips = liveClips.length > 0 ? liveClips : (job?.output?.clips || []);
if (clips.length === 0) return null;

// Job Detail Page
if (!job) return null;
```

Login Page Session Clear

```
// Clear any existing Google session on mount
useEffect(() => {
    googleLogout();
}, []);
```

Why? Prevents using cached Google account; user explicitly chooses account to use.

Error Recovery Patterns

Retry Mechanism

```
// Manual retry button
<Button onClick={() => window.location.reload()} variant="secondary">
    Retry
</Button>

// Automatic reconnect (WebSocket)
const timeout = Math.min(1000 * Math.pow(2, reconnectAttemptsRef.current), 10000);
setTimeout(connect, timeout);
```

Graceful Degradation

```
// usePages falls back to mock data
} catch (err) {
    setError(err instanceof Error ? err.message : 'Failed to load templates');
    // Fallback mock data for development
    const mockPages: Page[] = [
        { id: '1', name: 'Modern Minimal', ... },
        { id: '2', name: 'Bold Creator', ... },
    ];
    setPages(mockPages);
}
```

State Reset on Navigation

```
// Job detail page resets store before fetching new job
useEffect(() => {
    reset();  // Clear previous job state
    // ... fetch new job
}, [id]);
```

Performance and Loading States

> **Confidence:** CONFIRMED | **Source:** Direct code analysis

---

Loading State Patterns

Skeleton Screens

Used for initial data loading:

```
// JobsList skeleton
if (isLoading && items.length === 0) {
    return (
        <div className={styles.grid}>
            {[1, 2, 3, 4, 5, 6].map((i) => (
                <div key={i} className={styles.skeleton} />
            ))}
        </div>
    );
}

// TemplateSelector skeleton
if (isLoading) {
    return (
        <div className={styles.grid}>
            {[1, 2, 3, 4, 5, 6].map((i) => (
                <div key={i} className={styles.skeleton} />
            ))}
        </div>
    );
}
```

Spinner Loading

Used for focused actions:

```
// Job detail page loading
if (fetchStatus === 'loading') {
    return (
        <div className={styles.loading}>
            <div className={styles.spinner} />
            <p>Loading job details...</p>
        </div>
    );
}

// Auth guard loading
if (isLoading) {
    return (
        <div className="auth-loading">
            <div className="auth-loading-spinner" />
        </div>
    );
}
```

Button Loading State

```
<Button
    isLoading={state.isSubmitting}
    disabled={state.isSubmitting}
>
    {state.isSubmitting ? 'Creating Job...' : 'Create Job'}
</Button>
```

Inside Button component:

```
{isLoading && (
    <span className={styles.spinner}>
        <svg viewBox="0 0 24 24">
            <circle cx="12" cy="12" r="10" strokeDasharray="32">
                <animate attributeName="stroke-dashoffset" values="32;0" dur="1s" repeatCount="indefinite" />
            </circle>
        </svg>
    </span>
)}
```

Data Fetching Optimizations

Pages Cache

File: `src/hooks/usePages.ts`

```
// Module-level cache (survives component unmounts)
let pagesCache: Page[] | null = null;

export function usePages() {
    const [pages, setPages] = useState<Page[]>(pagesCache || []);
    const [isLoading, setIsLoading] = useState(!pagesCache);

    useEffect(() => {
        if (pagesCache) {
            setIsLoading(false);
            return;  // Skip fetch, use cache
        }

        async function fetchPages() {
            const data = await fetch(...);
            pagesCache = data;  // Cache for future
            setPages(data);
        }
        fetchPages();
    }, []);
}
```

Effect: Templates only fetched once per session.

Selective Zustand Subscriptions

```
// ✅ Only re-render when specific value changes
const job = useJobStore(state => state.job);
const liveClips = useJobStore(state => state.liveClips);

// ❌ Re-renders on ANY store change
const store = useJobStore();
```

Conditional WebSocket

```
const shouldConnect =
    fetchStatus === 'success' &&
    job &&
    job.status !== 'completed' &&
    job.status !== 'failed';

useJobWebSocket(shouldConnect ? id : '');
```

Effect: No WebSocket for completed/failed jobs. Saves server resources.

Polling Auto-Stop

```
useEffect(() => {
    const isTerminal = job?.status === 'completed' || job?.status === 'failed';
    if (isTerminal) return;  // Don't setup interval

    const intervalId = setInterval(fetchJob, 3000);
    return () => clearInterval(intervalId);
}, [job?.status, job?.phase]);
```

Effect: Polling stops when job reaches terminal state.

Animation Performance

GPU-Accelerated Transforms

```
// Uses transform instead of top/left
export const slideUpVariants: Variants = {
    initial: { opacity: 0, y: 20 },  // translateY
    animate: { opacity: 1, y: 0 },
};
```

Staggered List Rendering

```
export const staggerContainer: Variants = {
    animate: {
        transition: {
            staggerChildren: 0.05,  // 50ms between items
            delayChildren: 0.1,
        },
    },
};
```

Effect: Items animate one at a time, preventing frame drops from parallel animations.

Layout Animations

```
// Smooth sidebar/inspector size transitions
<motion.aside
    animate={{
        width: isInspectorOpen ? 'var(--inspector-width)' : 0,
        opacity: isInspectorOpen ? 1 : 0,
    }}
    transition={{ duration: 0.2 }}
>
```

Progress Indicators

Pipeline Timeline

Visual progress through job steps:

```
const STEPS = [
    { id: 'queued', label: 'Queued' },
    { id: 'downloading', label: 'Downloading' },
    { id: 'transcribing', label: 'Transcribing' },
    { id: 'analyzing', label: 'Analyzing' },
    { id: 'rendering', label: 'Rendering' },
];

// Determine which steps are done
const effectiveIndex = isCompleted ? STEPS.length : currentStepIndex;
const isDone = index < effectiveIndex;
```

Progress Bar

```
{['downloading', 'transcribing', 'analyzing', 'rendering'].includes(job.status) && (
    <div className={styles.progress}>
        <div
            className={styles.progressBar}
            style={{ width: `${job.progress}%` }}
        />
    </div>
)}
```

Memory Management

Ref Cleanup

```
// WebSocket cleanup on unmount
useEffect(() => {
    mountedRef.current = true;
    connect();

    return () => {
        mountedRef.current = false;
        if (wsRef.current) {
            wsRef.current.close(1000, 'Component unmounting');
            wsRef.current = null;
        }
        if (reconnectTimeoutRef.current) {
            clearTimeout(reconnectTimeoutRef.current);
        }
    };
}, [connect]);
```

Store Reset on Navigation

```
// Job detail page
useEffect(() => {
    reset();  // Clear previous job data
    fetchJob();
}, [id]);
```

Effect: Old job data doesn't pollute new job view.

Font Loading

File: `src/app/layout.tsx`

```
const inter = Inter({
    variable: "--font-sans",
    subsets: ["latin"],
    display: "swap",  // Show fallback font until loaded
});

const jetbrainsMono = JetBrains_Mono({
    variable: "--font-mono",
    subsets: ["latin"],
    display: "swap",
});
```

`display: "swap"`: Prevents invisible text during font load.

Environment and Configuration

> **Confidence:** CONFIRMED | **Source:** Direct code analysis of lib/config.ts

---

Environment Variables

Required Variables

Variable	Purpose	Default (Dev)	Example (Prod)
`NEXT_PUBLIC_API_URL`	REST API base URL	`http://localhost:8000`	`https://api.seone.io`
`NEXT_PUBLIC_WS_URL`	WebSocket base URL	`ws://localhost:8000`	`wss://api.seone.io`
`NEXT_PUBLIC_GOOGLE_CLIENT_ID`	Google OAuth client ID	`""`	`123456-abc.apps.googleusercontent.com`
`NEXT_PUBLIC_DATA_URL`	Media files URL	`http://localhost:8000/data`	`https://cdn.seone.io`

Environment File

Create `.env.local` in project root:

```
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_WS_URL=ws://localhost:8000
NEXT_PUBLIC_GOOGLE_CLIENT_ID=your-google-client-id.apps.googleusercontent.com
NEXT_PUBLIC_DATA_URL=http://localhost:8000/data
```

Note: File is gitignored for security.

Configuration Object

File: `src/lib/config.ts`

```
export const config = {
    api: {
        baseUrl: process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8000',
        version: 'v1',
        timeout: 30000,  // 30 seconds
    },

    ws: {
        baseUrl: process.env.NEXT_PUBLIC_WS_URL || 'ws://localhost:8000',
        reconnectAttempts: 5,
        reconnectDelay: 1000,
    },

    auth: {
        googleClientId: process.env.NEXT_PUBLIC_GOOGLE_CLIENT_ID || '',
        allowedDomain: ['creativefuel.io'],
        tokenCookieName: 'seone_token',
        tokenExpiry: 7,  // days
    },

    media: {
        dataBaseUrl: process.env.NEXT_PUBLIC_DATA_URL || 'http://localhost:8000/data',
    },
} as const;
```

URL Helpers

API URL

```
export const getApiUrl = (endpoint: string): string => {
    return `${config.api.baseUrl}${endpoint}`;
};

// Usage: getApiUrl('/api/v1/jobs') → 'http://localhost:8000/api/v1/jobs'
```

WebSocket URL

```
export const getWsUrl = (endpoint: string): string => {
    return `${config.ws.baseUrl}${endpoint}`;
};

// Usage: getWsUrl('/ws/jobs/123') → 'ws://localhost:8000/ws/jobs/123'
```

Media URL

```
export const getMediaUrl = (path: string): string => {
    const cleanPath = path.startsWith('/') ? path.slice(1) : path;
    return `${config.media.dataBaseUrl}/${cleanPath}`;
};

// Usage: getMediaUrl('/clips/job-id/clip_0.mp4')
//     → 'http://localhost:8000/data/clips/job-id/clip_0.mp4'
```

Endpoint Configuration

```
export const endpoints = {
    auth: {
        google: '/api/v1/auth/google',
        me: '/api/v1/auth/me',
        logout: '/api/v1/auth/logout',
    },
    jobs: {
        list: '/api/v1/jobs',
        create: '/api/v1/jobs',
        get: (id: string) => `/api/v1/jobs/${id}`,
        delete: (id: string) => `/api/v1/jobs/${id}`,
    },
    pages: {
        list: '/api/v1/pages',
        get: (id: string) => `/api/v1/pages/${id}`,
    },
    ws: {
        job: (jobId: string) => `/ws/jobs/${jobId}`,
    },
} as const;
```

Protocol Validation

On module load, validates WebSocket protocol:

```
function validateWsConfig(): void {
    if (typeof window === 'undefined') return;

    const isSecurePage = window.location.protocol === 'https:';
    const isSecureWs = config.ws.baseUrl.startsWith('wss://');

    if (isSecurePage && !isSecureWs) {
        console.error(
            '[FATAL CONFIG] Secure page (https) attempting non-secure WebSocket (ws://).\n' +
            `Current WS URL: ${config.ws.baseUrl}\n` +
            'Fix NEXT_PUBLIC_WS_URL to use wss:// in production.'
        );
    }

    if (process.env.NODE_ENV === 'production' && !isSecureWs) {
        console.warn('[CONFIG WARNING] WebSocket URL is not secure (wss://).');
    }
}

validateWsConfig();
```

Why? Mixed content (HTTPS page + WS endpoint) fails silently. This catches misconfiguration early.

TypeScript Configuration

File: `tsconfig.json`

```
{
    "compilerOptions": {
        "target": "ES2017",
        "strict": true,
        "jsx": "react-jsx",
        "moduleResolution": "bundler",
        "paths": {
            "@/*": ["./src/*"]
        }
    }
}
```

Key Settings: - `strict: true` — Full type checking - `@/*` path alias — Import from `@/components` instead of `../../components`

Next.js Configuration

File: `next.config.ts`

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
    /* config options here */
};

export default nextConfig;
```

Currently using defaults. Potential additions: - `images.domains` for external image hosts - `rewrites` for API proxying - `experimental` features

Accessibility and UX Decisions

> **Confidence:** INFERRED | Some accessibility features may need verification

---

Accessibility Features

Focus States

File: `src/app/globals.css`

```
:focus-visible {
    outline: 2px solid var(--accent-primary);
    outline-offset: 2px;
}
```

All interactive elements show a visible focus ring when navigated via keyboard.

Semantic HTML

Components use appropriate semantic elements:

```
// Navigation
<nav className={styles.nav}>...</nav>

// Headers
<header className={styles.topbar}>...</header>

// Main content
<main className={styles.canvas}>...</main>

// Aside panels
<aside className={styles.inspector}>...</aside>
```

Button Accessibility

```
<button
    className={styles.menuButton}
    onClick={toggleSidebar}
    aria-label="Toggle menu"  // Screen reader label
>
    <MenuIcon />
</button>

<button
    aria-label="Toggle inspector"
    title={isInspectorOpen ? 'Hide Inspector' : 'Show Inspector'}
>
    <PanelIcon />
</button>
```

Form Labels

```
<Input
    label="YouTube URL"  // Generates <label> with for attribute
    id={inputId}         // Links label to input
/>
```

Internal implementation:

```
const inputId = id || `input-${Math.random().toString(36).slice(2, 9)}`;

{label && (
    <label htmlFor={inputId} className={styles.label}>
        {label}
    </label>
)}
<input ref={ref} id={inputId} ... />
```

Dark Mode

The application uses a dark theme by default:

```
// src/app/layout.tsx
<html lang="en" className="dark">
```

```
/* Color scheme declaration */
html {
    color-scheme: dark;
}

body {
    background: var(--bg-primary);  /* #0a0a0b */
    color: var(--text-primary);      /* #ffffff */
}
```

Rationale: Video editing tools typically use dark themes to reduce eye strain and make content preview more accurate.

Text Rendering

```
html {
    -webkit-font-smoothing: antialiased;
    -moz-osx-font-smoothing: grayscale;
    text-rendering: optimizeLegibility;
}
```

Effect: Smoother, more readable text across browsers.

Reduced Motion

Status: NOT IMPLEMENTED

Currently, animations do not respect `prefers-reduced-motion`. Recommended addition:

```
@media (prefers-reduced-motion: reduce) {
    *, *::before, *::after {
        animation-duration: 0.01ms !important;
        transition-duration: 0.01ms !important;
    }
}
```

UX Decisions

Loading Feedback

Action	Feedback
Initial page load	Skeleton screens
Button submission	Spinner + disabled state
Job fetch	Centered spinner with text
Auth check	Full-screen spinner

Error Communication

Error Type	Display
Form validation	Inline below field
API error	Alert/toast + error message
403/404	Full page with back button
Session expiry	Alert + redirect

Progress Visualization

Pipeline Timeline: - Visual steps with checkmarks for completed - Current step highlighted - Failed step shows X with error message

Progress Bar: - Only shown during active processing - Width based on `clips_ready / clip_count`

Navigation Patterns

Sidebar Active State: - Current page highlighted with accent color - Animated indicator using `layoutId`

Breadcrumb-style Back: - Job detail page has "← Back" button - Returns to jobs list

Confirmation Patterns

Action	Confirmation
Logout	Immediate (no confirmation)
Create job	Form validation, then submit
Delete job	NOT IMPLEMENTED

Mobile Responsiveness

Sidebar Behavior

```
// Mobile: sidebar hidden by default, overlay when open
{isSidebarOpen && (
    <motion.div
        className={styles.overlay}
        onClick={() => setSidebarOpen(false)}
    />
)}

// Toggle via hamburger menu in TopBar
<button className={styles.menuButton} onClick={toggleSidebar}>
    <MenuIcon />
</button>
```

CSS Variables for Layout

```
:root {
    --sidebar-width: 260px;
    --inspector-width: 340px;
    --topbar-height: 60px;
}
```

These can be adjusted via media queries for responsive layouts.

Color Contrast

Usage	Foreground	Background	Ratio (approx)
Primary text	#ffffff	#0a0a0b	21:1 ✓
Secondary text	#a1a1a6	#0a0a0b	7:1 ✓
Muted text	#6e6e73	#0a0a0b	4:1 ⚠
Accent on dark	#6366f1	#0a0a0b	4.5:1 ✓

Note: Muted text may not meet WCAG AA for small text. Consider for future improvement.

Known Limitations and Risks

> **Confidence:** CONFIRMED + INFERRED | Based on code analysis and CONTRACTS.md

---

Known Limitations

1. No Token Refresh

Issue: JWT tokens are not automatically refreshed.

Current Behavior: - Token expires after 30 minutes (backend default) - On 401, user is redirected to login - User must re-authenticate

Impact: Users in long sessions will be interrupted.

Mitigation Options: - Implement refresh token flow - Extend token expiry - Show warning before expiry

---

2. No Offline Support

Issue: Application requires constant network connection.

Current Behavior: - No service worker - No caching of API responses - WebSocket disconnects are handled but offline state is not

Impact: Users lose all functionality if network drops.

---

3. Session Storage Only

Issue: Auth state uses sessionStorage, not localStorage.

Current Behavior: - Opening new browser window requires re-login - Closing browser logs user out

Impact: May be unexpected for users used to persistent sessions.

Note: This is an intentional security decision, not a bug.

---

4. No Job Deletion UI

Issue: Delete endpoint exists but no UI to trigger it.

Endpoints defined:

```
jobs: {
    delete: (id: string) => `/api/v1/jobs/${id}`,
}
```

Current Behavior: Users cannot delete jobs from the UI.

---

5. Hardcoded Domain Restriction

Issue: Allowed domains are hardcoded in config.

```
auth: {
    allowedDomain: ['creativefuel.io'],
}
```

Impact: Adding new allowed domains requires code change and redeploy.

Mitigation: Move to environment variable or backend-controlled list.

---

6. Mock Data Fallback

Issue: Template selector falls back to mock data on error.

```
} catch (err) {
    // Fallback mock data for development
    const mockPages: Page[] = [
        { id: '1', name: 'Modern Minimal', ... },
    ];
    setPages(mockPages);
}
```

Risk: If API fails in production, users see development mock data.

Mitigation: Remove fallback or use production-appropriate error handling.

---

7. No Rate Limiting UI

Issue: No visual feedback if user hits rate limits.

Current Behavior: Generic error message displayed.

Impact: Users don't know why requests are failing or when to retry.

Potential Race Conditions

1. REST/WebSocket Race

Scenario: WebSocket event arrives before REST response.

Current Handling: Both update same store; last write wins.

Risk Level: Low — state converges quickly via polling.

2. Navigation During Fetch

Scenario: User navigates away while job is being fetched.

Current Handling: `mountedRef` prevents state updates after unmount.

```
if (!mountedRef.current) return;
```

Risk Level: Low — handled correctly.

3. Rapid Job Navigation

Scenario: User clicks between jobs quickly.

Current Handling: `reset()` clears previous job state; new fetch begins.

Risk Level: Low — but may show flash of previous job.

Security Considerations

1. Client-Side Domain Check

Current: Domain validation happens on frontend before API call.

```
if (!isAllowedDomain(payload.email)) {
    setError('Only @xyz allowed');
    return;
}
```

Risk: Client-side checks can be bypassed.

Mitigation: Backend ALSO validates domain. Client-side is UX only.

2. JWT in Query Parameter

Current: WebSocket auth uses query parameter.

```
const wsUrl = `...?token=${encodeURIComponent(token)}`;
```

Risk: Token may appear in server logs.

Mitigation: - Use short-lived tokens - Backend should support header-based auth for WS - Ensure logs are secured

3. Non-httpOnly Cookie

Current: JWT cookie is readable by JavaScript.

```
Cookies.set(config.auth.tokenCookieName, token, {
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'strict',
    // httpOnly: NOT SET
});
```

Risk: XSS could steal tokens.

Mitigation: - Strict CSP headers - Input sanitization - Consider httpOnly with server-side cookie setting

---

Missing Features

Feature	Status	Notes
Toast notifications	⬜ Missing	Currently uses `alert()`
Job cancellation	⬜ Missing	No cancel button/endpoint
Job deletion	⬜ Missing	Endpoint exists, no UI
Clip download all	⬜ Missing	Individual downloads only
Search/filter jobs	⬜ Missing	Only pagination
User settings	⬜ Missing	No preferences page
Dark/light toggle	⬜ Missing	Dark mode only
Reduced motion	⬜ Missing	Animations always play
Error boundary	⬜ Missing	Crashes propagate up
Sentry/logging	⬜ Missing	Console.error only

Browser Compatibility

Tested

- Chrome (latest)
- Firefox (latest)
- Safari (latest)

Potential Issues

- **IE11:** Not supported (ES6+, CSS variables)
- **Old Safari:** WebSocket may have quirks
- **Mobile browsers:** Sidebar overlay behavior

Performance Concerns

1. Large Job Lists

Issue: All jobs loaded into memory for pagination.

Mitigation: Use proper pagination with page limits.

2. Many Clips

Issue: ClipGallery renders all clips with `<video>` elements.

Risk: Memory usage for jobs with many clips.

Mitigation: Virtual scrolling for large clip counts.

3. WebSocket Memory

Issue: Events not garbage collected if component stays mounted.

Current: Events are processed immediately, not stored.

Risk Level: Low — no event history maintained.

Recommendations for Future

1. **Implement token refresh** — Most impactful for user experience
2. **Add error boundary** — Prevent full app crashes
3. **Implement toast system** — Replace alerts with non-blocking notifications
4. **Add job deletion** — Complete CRUD operations
5. **Implement reduced motion** — Accessibility improvement
6. **Add loading states for clips** — Show skeleton while video loads
7. **Consider httpOnly cookies** — Enhanced security